The *down()* in *my_write()* function acquires the semaphore *test_Semaphore*. If no more tasks can acquire the semaphore, calling this function will put the task to sleep until the semaphore is released. *up()* is used to release the semaphore. Unlike mutexes, *up()* may be called from any context and even by tasks which have never called *down()*.

In Line 62, *sem_init()* initialises *test_semaphore* dynamically at run time.

rt-mutex.c: This complete code is the same as that of *withoutLock.c* except for some changes mentioned below.

DEFINE_RT_MUTEX defines and initialises rt-mutex. It is useful to define global rt-mutexes.

rt_mutex_lock (line 24): Locks rt-mutex.

rt_mutex_unlock (line 28): Unlocks rt_mutex

rt_mutex_init (line 62): Initialises the *rt* lock to the unlocked state. Initialising of a locked *rt* lock is not allowed.

rt_mutex_destroy (line 69): This function marks the mutex as uninitialised, and any subsequent use of the mutex is forbidden. The mutex must not be locked when this function is called.

Compiling the kernel module

To compile the module, refer to the *makefile* available at the GitHub link. After compiling and loading this module, the output of the *printk* statement in the *init_module* is displayed. The major number allocated for *mydevice* can be seen in the output; if not, check the */proc/devices* file, where it can be seen in the character device list. In my system, a major number 241 was allocated to *mydevice*. This module only registers the device. You also need to create a device file for accessing the device. You can create the device file in the present working directory using the following command:

```
mknod mydevice c 241 0
```

Application programs: low.c, middle.c and high.c

Lines 2 to 13 represent header files. A call to *open()* at Line 22 creates a new open file description, an entry in the system-wide table of open files. This entry records the file offset. A file descriptor is a reference to one of these entries.

The standard Linux kernel provides two real-time scheduling policies, *SCHED_FIFO* and *SCHED_RR*. The main real-time policy is *SCHED_FIFO*. It implements a first-in, first-out scheduling algorithm. When a *SCHED_FIFO* task starts running, it continues to run until it voluntarily yields the processor, blocks it or is pre-empted by a higher-priority real-time task. It has no time-slices. All other tasks of lower priority will not be scheduled until it relinquishes the CPU. Two equal priority *SCHED_FIFO* tasks do not pre-empt each other. *SCHED_RR* is like *SCHED_FIFO*, except that such tasks are allotted time-slices based on their priority and run until they exhaust their time-slice. Non-real-time tasks use the *SCHED_NORMAL* scheduling policy (older kernels had a policy named *SCHED_OTHER*).

For this experiment, we have considered the FIFO scheduling policy (Lines 25 - 37).

In the standard Linux kernel, real-time priorities range from zero to *MAX_RT_PRIO-1*, inclusive. By default, *MAX_RT_PRIO* is 100. Non-real-time tasks have priorities in the range of *MAX_RT_PRIO* to (*MAX_RT_PRIO + 40*). These constitute the nice values of *SCHED_NORMAL* tasks. By default, the –20 to 19 nice range maps directly onto the priority range of 100 to 139.

Note: Root privileges are needed to choose schedulers. So, before proceeding to the execution, make sure that you are executing the code with the root login.

Write() at Line 42 writes up to *sizeof(bufw)* bytes from the buffer, starting at *bufw* to the file referred to by the file descriptor *fd*.

.c, m.c and h.c: All the code descriptions are the same as mentioned above, except for CPU affinity. These application programs are bound to get executed on a single CPU with the help of CPU affinity.

void CPU_ZERO (*cpu_set_t *set*): This macro initialises the CPU set to be the empty set.

*void CPU_SET (int cpu, cpu_set_t *set*): This macro sets the CPU affinity to *cpu*. The *cpu* parameter must not have side effects since it is evaluated more than once.

*int sched_setaffinity (pid_t pid, size_t cpusetsize, const cpu_set_t *cpuset)*: *sched_setaffinity* ( ) system call is used to pin the given process to the specified processor(s). You can get more information from man 2 *sched_setaffinity*. If

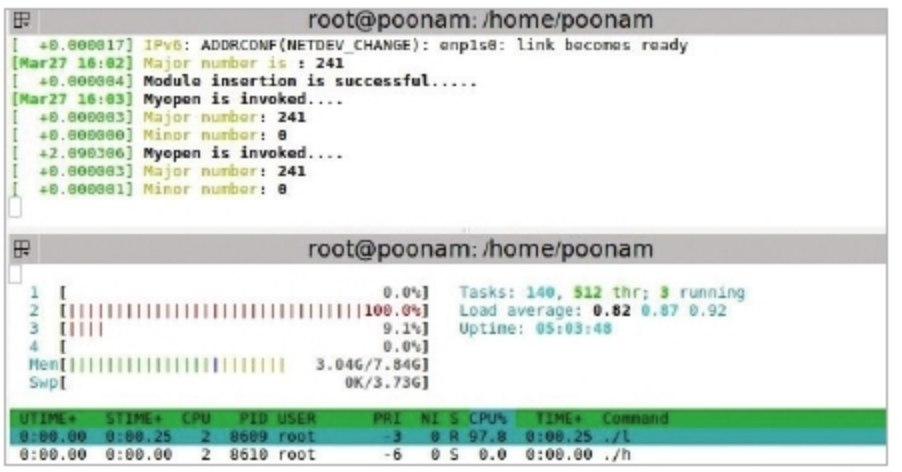

Figure 3: A low priority process is running

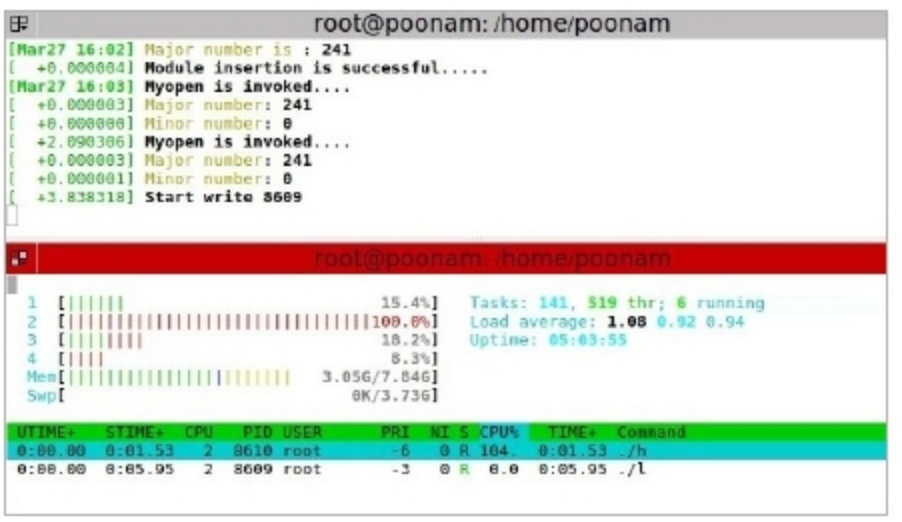

Figure 4: The high priority process pre-empts the low priority process